

Basic (your first graph) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `first_graph_practice.py` : this is the smallest possible LangGraph graph — one field, two nodes, one edge — deliberately tiny so the 3 real ingredients (state, nodes, edges) are visible on their own, before Doc07's whole agent loop gets rebuilt as a graph in the next exercise. **If not:** `agent_loop_to_graph` would be the first place you ever saw `StateGraph` / `add_node` / `add_edge` at all, with a much bigger graph to learn them from.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — `set_entry_point` / `set_finish_point`

```
# first_graph_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph

class GraphState(TypedDict):
    message: str

def node_one(state):
    return {"message": "Hello"}

def node_two(state):
    return {"message": state["message"] + ", World!"}

builder = StateGraph(GraphState)
builder.add_node("node_one", node_one)
builder.add_node("node_two", node_two)
builder.add_edge("node_one", "node_two")
builder.set_entry_point("node_one")
builder.set_finish_point("node_two")
```

```
graph = builder.compile()
result = graph.invoke({"message": "start"})
print(result)
```

Expected output:

```
{'message': 'Hello, World!'}
```

`set_entry_point("node_one")` and `set_finish_point("node_two")` are convenience methods — under the hood they're doing exactly `add_edge(START, "node_one")` and `add_edge("node_two", END)`, just without you importing `START / END` yourself. This version works correctly and traces exactly how you'd expect by hand: `node_one` sets `message` to `"Hello"`, then `node_two` reads that and appends `", World!"`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — explicit `START / END` , with type hints

```
# first_graph_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    message: str

def node_one(state: GraphState) -> dict:
    return {"message": "Hello"}

def node_two(state: GraphState) -> dict:
    return {"message": state["message"] + ", World!"}

builder = StateGraph(GraphState)
builder.add_node("node_one", node_one)
builder.add_node("node_two", node_two)
builder.add_edge(START, "node_one")
builder.add_edge("node_one", "node_two")
builder.add_edge("node_two", END)

graph = builder.compile()
result = graph.invoke({"message": "start"})
print(result)
```

Expected output:

```
{'message': 'Hello, World!'}
```

Difference from Basic: same graph, same result — this version just spells out the entry and exit as real edges (`add_edge(START, "node_one")` , `add_edge("node_two", END)`) instead of using the `set_entry_point` / `set_finish_point` shortcuts, and adds type hints (`state: GraphState` , `->`

`dict`) to both node functions. Being explicit about `START / END` matters more once a graph has more than one possible entry or exit point — a shortcut method can only point at one node at a time, while `add_edge(START, ...)` can be called more than once if you ever need multiple entry nodes.

Approach 2 — a second field with a reducer

Story: by default, LangGraph merges a node's returned dict one key at a time — for a plain field, that's an overwrite. The moment two different nodes both need to append to the *same* running log, an overwrite silently erases whatever the earlier node wrote. **If not:** the scratchpad you build in the very next exercise (`agent_loop_to_graph`) would lose every step but the last one, with no error telling you why.

```
# first_graph_practice.py
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    message: str
    # why: operator.add on a list means concatenate, not overwrite
    log: Annotated[list[str], operator.add]

def node_one(state: GraphState) -> dict:
    return {"message": "Hello", "log": ["node_one ran"]}

def node_two(state: GraphState) -> dict:
    return {"message": state["message"] + ", World!", "log": ["node_two ran"]}

builder = StateGraph(GraphState)
builder.add_node("node_one", node_one)
builder.add_node("node_two", node_two)
builder.add_edge(START, "node_one")
builder.add_edge("node_one", "node_two")
builder.add_edge("node_two", END)

graph = builder.compile()
result = graph.invoke({"message": "start", "log": []})
print(result)
```

Expected output:

```
{'message': 'Hello, World!', 'log': ['node_one ran', 'node_two ran']}
```

Notice `log` ends up with **both** entries, in order — `node_two` 's returned `{"log": ["node_two ran"]}` didn't overwrite `node_one` 's write, because `operator.add` (list concatenation) ran instead of the default plain overwrite. `message` still just overwrites, because it has no reducer annotation — proving both merge behaviors side by side in the same state shape.

Difference from Approach 1: Approach 1's single `message` field only ever needs (and only ever gets) the default overwrite behavior. This version adds a second field, `log`, annotated `Annotated[list[str], operator.add]`, so `LangGraph` calls `operator.add(old_log, new_log)` to merge instead of replacing — the one piece of machinery you need the moment a field should accumulate a history instead of holding a single current value.

Which one should you actually write? For a graph this small, Basic's `set_entry_point / set_finish_point` is perfectly fine and reads cleanly. Once a graph gets more than a couple of nodes, or you can imagine adding a second entry point later, switch to Approach 1's explicit `START / END` edges — it's the form you'll see in almost every real `LangGraph` example, so it's worth being fluent in it early. Reach for Approach 2's reducer pattern the moment any field in your state needs to grow instead of being replaced — a running log, a list of tool calls, a list of sources found — which is most real graphs past this first exercise.